

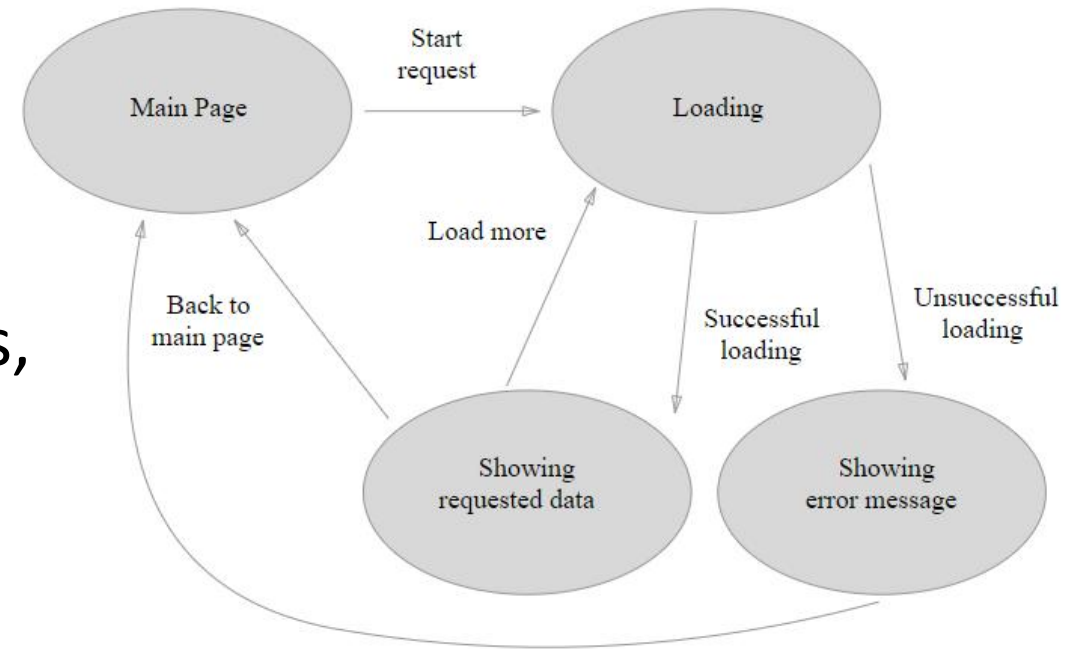
Mastering Embedded Systems A Practical Approach

Lesson 0.1: FSM

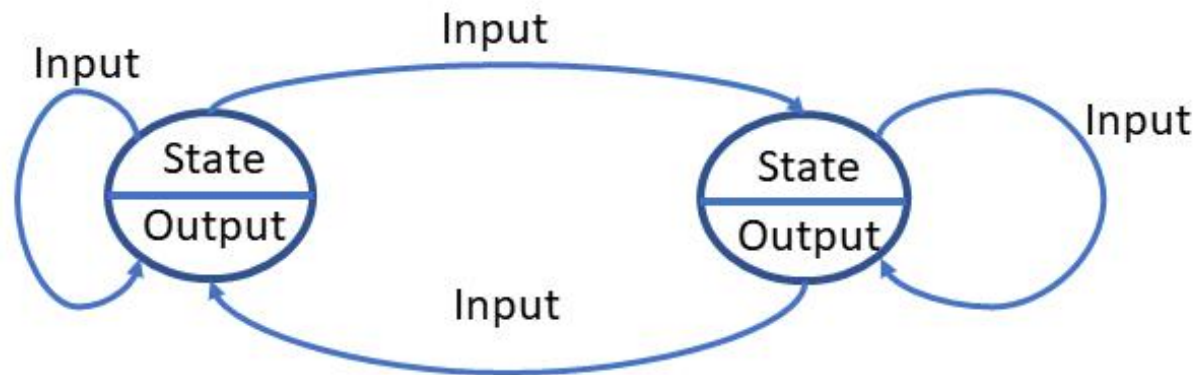
| Finite State Machines

| FSM - Introduction

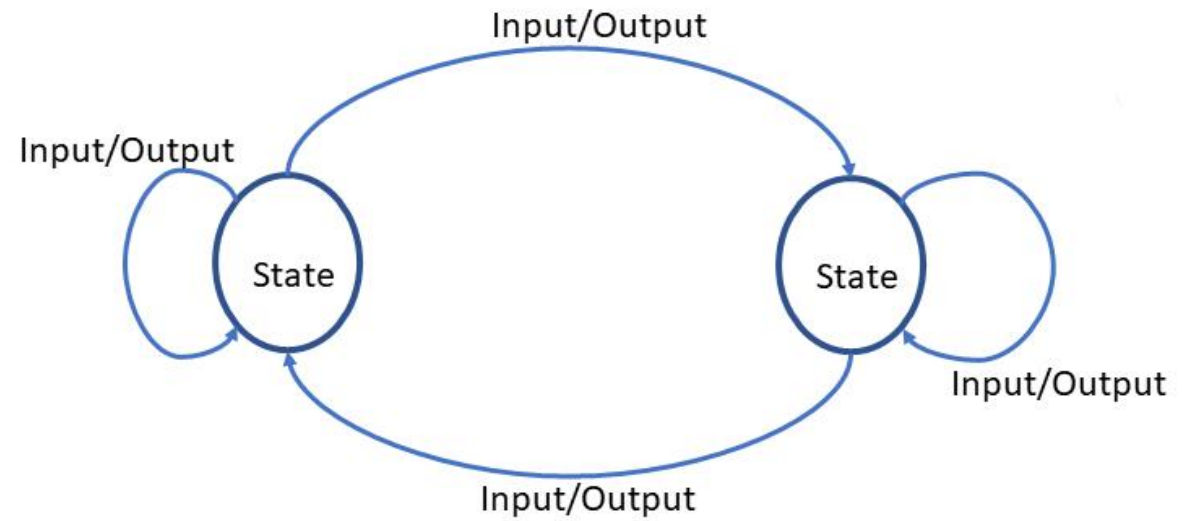
- FSMs are mathematical models used to represent and control the behavior of systems.
- They consist of a finite number of states, transitions between these states, and events that trigger transitions



| FSM - Introduction



Generic Moore model



Generic Mealy model

```

#include <stdio.h>

// Enumeration of states
enum States {
    STATE_A,
    STATE_B,
    STATE_C
};

// Function to implement Moore Machine logic
void mooreMachine(int inputEvent) {
    // Current state variable
    static enum States currentState = STATE_A;
    // State transitions
    switch (currentState) {
        case STATE_A:
            if (inputEvent == 1) {currentState = STATE_B;}
            break;
        case STATE_B:
            if (inputEvent == 2) {currentState = STATE_C;}
            break;
        case STATE_C:
            if (inputEvent == 3) {currentState = STATE_A;}
            break;
        // Handle additional states and transitions as needed
    }

    // Output based on the current state
    switch (currentState) {
        case STATE_A:
            printf("Current State: A. Output: Red light.\n");
            break;
        case STATE_B:
            printf("Current State: B. Output: Green light.\n");
            break;
        case STATE_C:
            printf("Current State: C. Output: Yellow light.\n");
            break;
    }
}

int main() {
    // Example input events
    // Triggering transition from STATE_A to STATE_B
    mooreMachine(1);
    // Triggering transition from STATE_B to STATE_C
    mooreMachine(2);
    // Triggering transition from STATE_C to STATE_A
    mooreMachine(3);
    return 0;
}

```

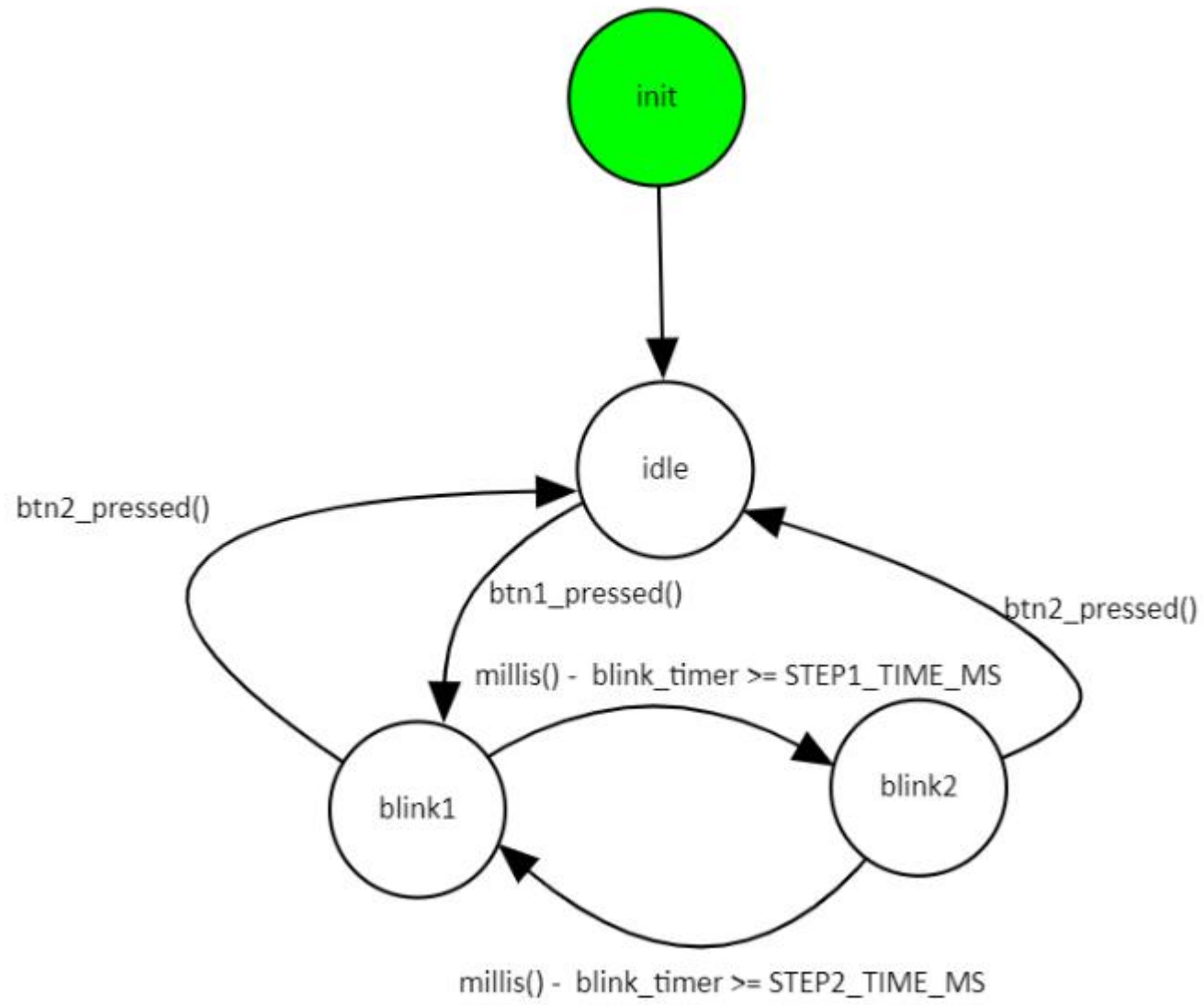
```

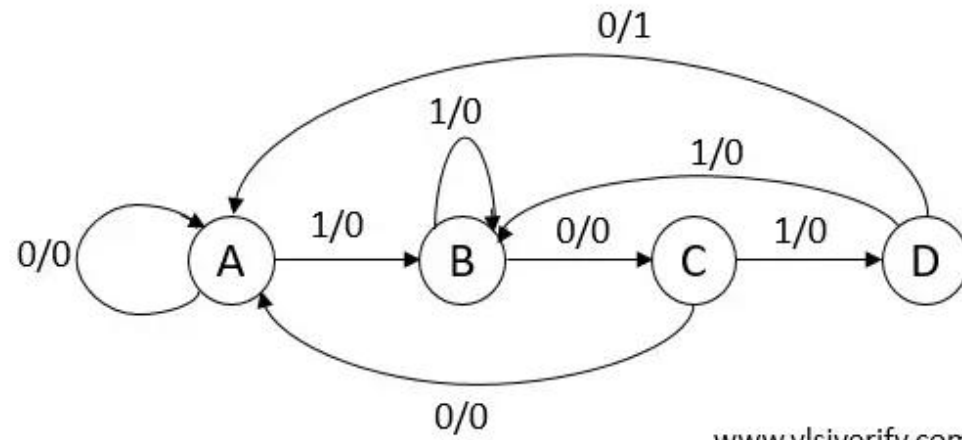
#include <stdio.h>
// Enumeration of states
enum States {
    STATE_A,
    STATE_B,
    STATE_C
};
// Function to implement Mealy Machine logic
void mealyMachine(int inputEvent) {
    // Current state variable
    static enum States currentState = STATE_A;
    // State transitions and outputs
    switch (currentState) {
        case STATE_A:
            if (inputEvent == 1) {
                currentState = STATE_B;
                printf("Transitioned to STATE_B. Output: Red light.\n");
            }
            break;
        case STATE_B:
            if (inputEvent == 2) {
                currentState = STATE_C;
                printf("Transitioned to STATE_C. Output: Green light.\n");
            }
            break;
        case STATE_C:
            if (inputEvent == 3) {
                currentState = STATE_A;
                printf("Transitioned to STATE_A. Output: Yellow light.\n");
            }
            break;
        // Handle additional states and transitions as needed
    }
}

int main() {
    // Example input events
    mealyMachine(1); // Triggering transition from STATE_A to STATE_B
    mealyMachine(2); // Triggering transition from STATE_B to STATE_C
    mealyMachine(3); // Triggering transition from STATE_C to STATE_A

    return 0;
}

```





www.vlsiverify.com

1010 Non-Overlapping Mealy Sequence Detector

| Exercises 01

Design the FSM, Flowchart, and code for a program to detect '10110' in a string of 1s and 0s that is input.

| Exercises 02

Design FSM and Develop the firmware to support
“Pressing the button will change the status of the led:
Debouncing a button with interrupt. Debounce time is 50ms”

The background is a solid dark blue color. It is decorated with a repeating pattern of white, isometric cubes. These cubes are arranged in a staggered, three-dimensional grid that covers the entire surface. The perspective is from an angle, giving the cubes a sense of depth and volume. The lines of the cubes are thin and white, creating a high-contrast geometric design.

*Let's
Innovate Together*